COL783 Assignment 1 – Report

2023CS10807 Samyak Sanghvi
2021CS50602 Kashish Goel

1 Part 1: Geometric Transformation

1.1 A. Method (Corner selection, rotation, homography)

Rotation (alignment). Let the selected corners be $\{\mathbf{p}_i = (x_i, y_i)\}_{i=1}^4$ ordered $(\text{tl}, \text{tr}, \text{br}, \text{bl})$. Define top and bottom midpoints

$$\mathbf{m}_t = \tfrac{1}{2}(\mathbf{p}_{\text{tl}} + \mathbf{p}_{\text{tr}}), \qquad \mathbf{m}_b = \tfrac{1}{2}(\mathbf{p}_{\text{bl}} + \mathbf{p}_{\text{br}}),$$

the vector $\mathbf{v} = \mathbf{m}_b - \mathbf{m}_t$, and its angle $\theta = \text{atan2}(v_y, v_x)$. To make the document vertical, rotate by $\alpha = \frac{\pi}{2} - \theta$ with

$$R(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix}.$$

If $\mathbf{c} = (c_x, c_y)$ is the image center, the affine mapping is $\mathbf{x}' = R(\alpha)(\mathbf{x} - \mathbf{c}) + \mathbf{c}$.

Perspective transform A 3×3 transformation matrix (homography) H maps homogeneous coordinates $\tilde{\mathbf{x}} = [x, y, 1]^T$ to $\tilde{\mathbf{X}} = H\tilde{\mathbf{x}}$ with Cartesian $(X, Y) = (\tilde{X}/\tilde{w}, \tilde{Y}/\tilde{w})$. With $H = [h_{ij}]$, each correspondence $(x, y) \mapsto (X, Y)$ satisfies

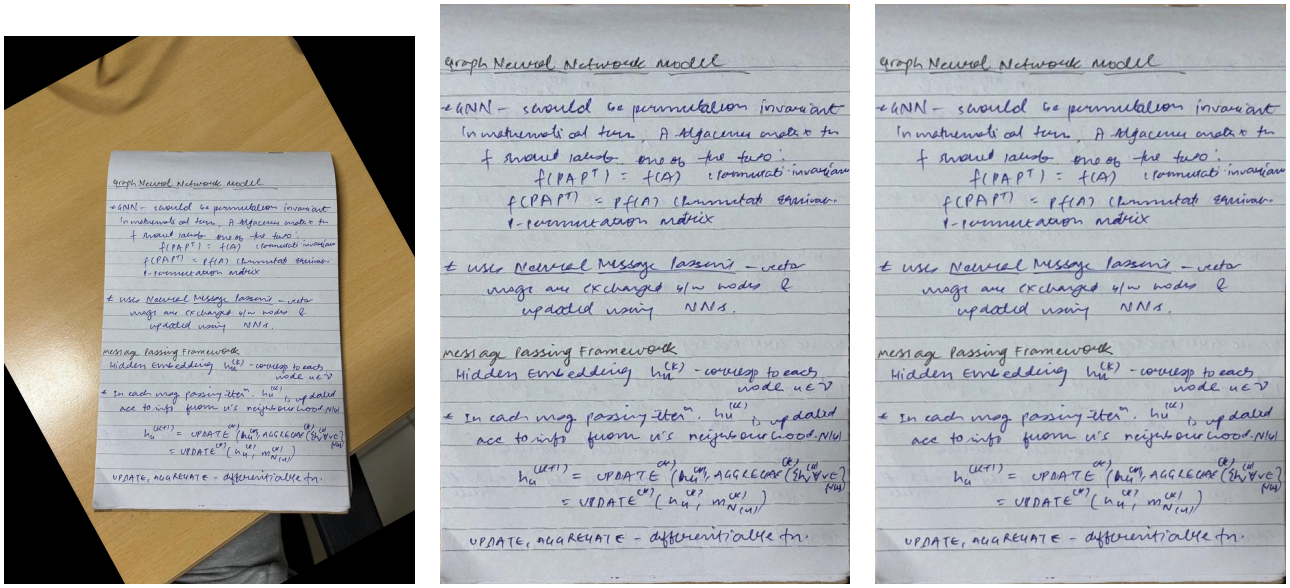
$$X = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \qquad Y = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}.$$

Four point pairs (the document corners to rectangle corners) give eight equations to solve H up to scale. Warping uses the inverse mapping $\tilde{\mathbf{x}} \sim H^{-1}\tilde{\mathbf{X}}$ and resampling (nearest or bilinear).

1.2 B. Results (Nearest vs Bilinear)

Resampling formulas. For a source location (u, v) with $i = \lfloor u \rfloor$, $j = \lfloor v \rfloor$, $\alpha = u - i$, $\beta = v - j$:

- Nearest neighbour: $I'(X, Y) = I(\text{round}(u), \text{round}(v))$.
- Bilinear: $I'(X, Y) = (1-\alpha)(1-\beta)I_{i,j} + \alpha(1-\beta)I_{i+1,j} + (1-\alpha)\beta I_{i,j+1} + \alpha\beta I_{i+1,j+1}$.



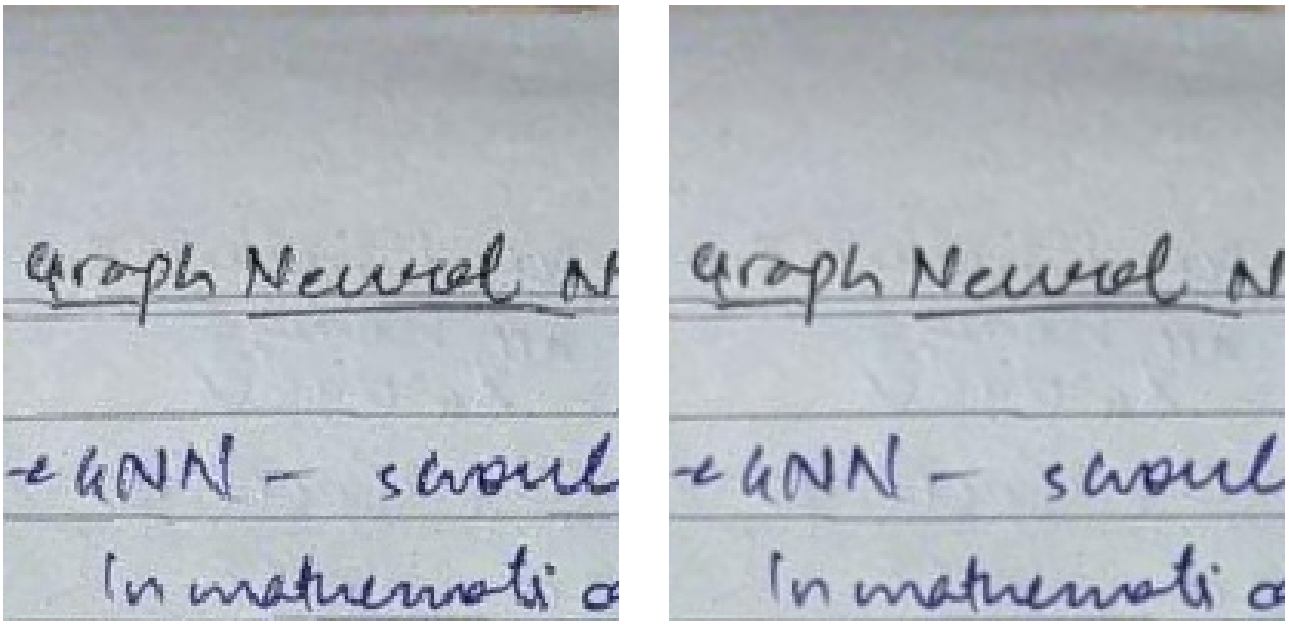
(a) Rotated (b) Scanned (Nearest) (c) Scanned (Bilinear)

Figure 1: Part 1: Rotation and perspective transform with two interpolation schemes.

(Interpolation flags used in the code: `cv2.INTER_NEAREST` for nearest-neighbour, `cv2.INTER_LINEAR` for bilinear.)

1.3 C. Zoomed comparisons

Zoom crop. The zoomed views are simple subimages: for height/width 200, $I_{\text{zoom}} = I[0{:}200, 0{:}200]$ (row, column slice).



(a) Zoomed (Nearest)

(b) Zoomed (Bilinear)

Figure 2: Nearest is sharper but jaggy; bilinear is smoother with fewer aliasing artifacts.

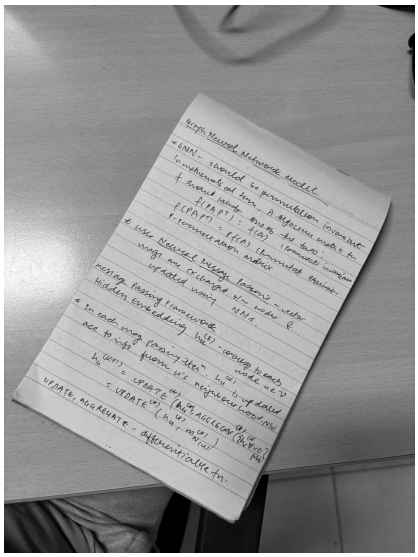
(The zoom images are produced by cropping the top-left 200×200 pixels of the warped images in the code: `zoom = warped[:200,:200]`.)

2 Part 2: Intensity Transformations

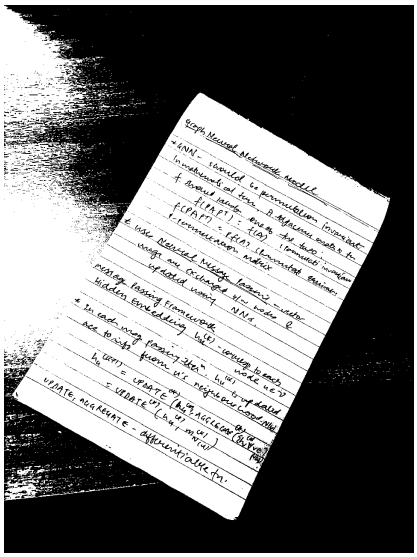
All operations act on a grayscale image $I \in [0, 255]$.

2.1 A. Manual threshold

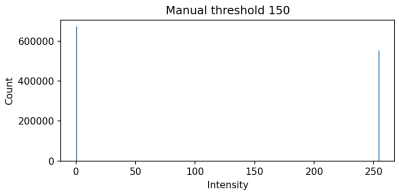
Binary output $O = \mathbb{1}[I \geq T] \cdot 255$. Here $T = 150$.



(a) Original



(b) Threshold 150



(c) Histogram

2.2 B. Linear transform

Manual: $O = \text{clip}(aI + b, 0, 255)$ with $a = 1.9, b = -300$. Auto: percentiles map $[p_{\text{low}}, p_{\text{high}}] \to [0, 255]$.

Manual linear stretch. The linear transform rescales intensity via an affine mapping $I \mapsto aI + b$ followed by clipping to $[0, 255]$. Increasing the slope $a > 1$ increases contrast by spreading values away from mid-gray; the bias b recenters the mapped values. For this image we found $(a, b) = (1.9, -300)$ by trial — this brightens the background (mapped toward 255) while pushing ink pixels toward 0, producing a near-binary visual appearance without explicit thresholding.

Automatic parameter selection (percentile-based contrast stretch). A robust automatic way to set a and b is to compute two percentiles of the input histogram, p_{low} and p_{high} (for example the 2nd and 98th percentiles), and linearly map that interval to $[0, 255]$. Percentiles avoid extreme outliers (single very dark or very bright pixels) from dominating the stretch.

Given p_{low} and p_{high} (intensity values in $[0, 255]$), compute

$$a = \frac{255}{p_{\text{high}} - p_{\text{low}}}, \qquad b = -a \cdot p_{\text{low}}.$$

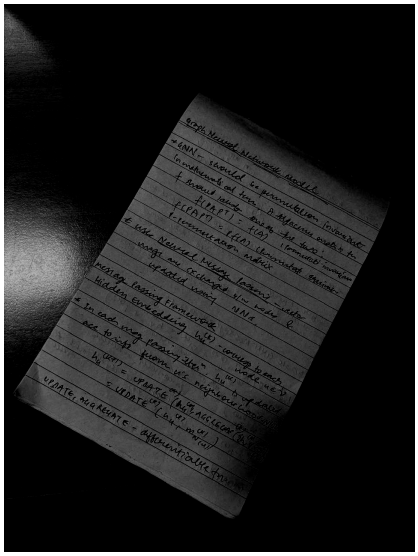
Then apply $O = \text{clip}(aI + b, 0, 255)$.

Why percentiles? Using percentiles (instead of min and max) prevents a single speck of dirt or a specular highlight from forcing an extreme mapping that would crush contrast. The lower percentile sets a robust black reference; the upper percentile sets a robust white reference. Typical choices are $(p_{\text{low}}, p_{\text{high}}) = (2\%, 98\%)$ or $(1\%, 99\%)$; the exact choice trades off clipping more pixels (wider percentiles) vs. preserving extreme values (narrower percentiles).

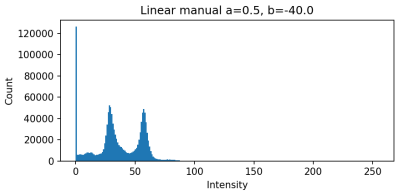
Pseudocode (auto stretch)

```
# I is the grayscale image
p_low  = percentile(I, p_low_percent)   # e.g., 2
p_high = percentile(I, p_high_percent)  # e.g., 98
if p_high == p_low:
    # fallback (flat image): use small slope or skip
a = 255.0 / (p_high - p_low)
b = -a * p_low
O = clip(a * I + b, 0, 255).astype(uint8)
```

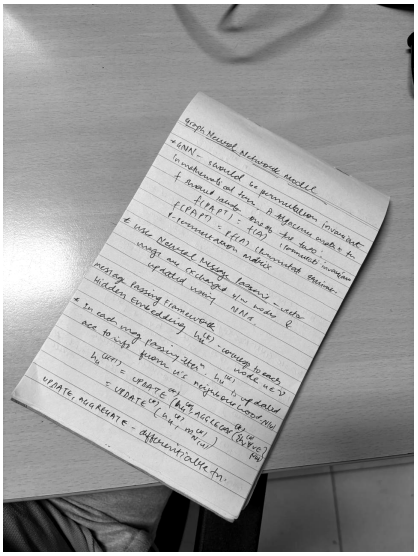
Observed result. In our experiments the automatic percentile mapping (we used the 2nd and 98th percentiles in the provided figure c below) produces a clean separation of paper and ink while being robust to a few bright/dark outliers. Compared to the manual (a, b) that was tuned visually, the percentile method gives a reproducible automatic result that generalizes across other page photographs.



(a) Manual (a=1.9, b=−300)



(b) Histogram

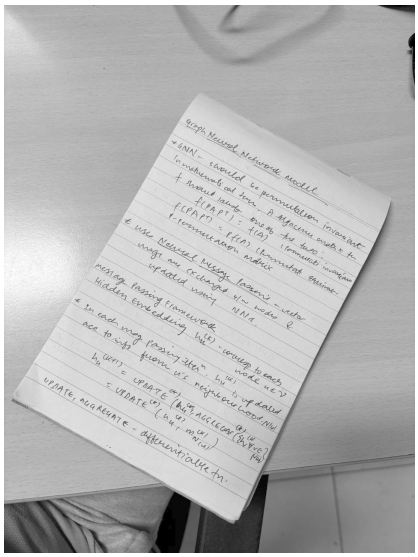


(c) Auto linear

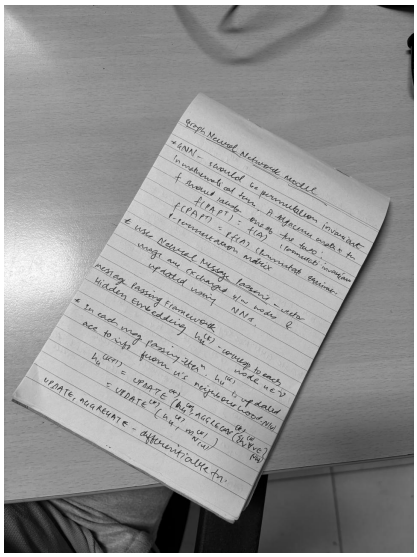
2.3 C. Gamma correction

$O = 255\,(I/255)^{\gamma}$. γ<1 brightens; γ>1 darkens.

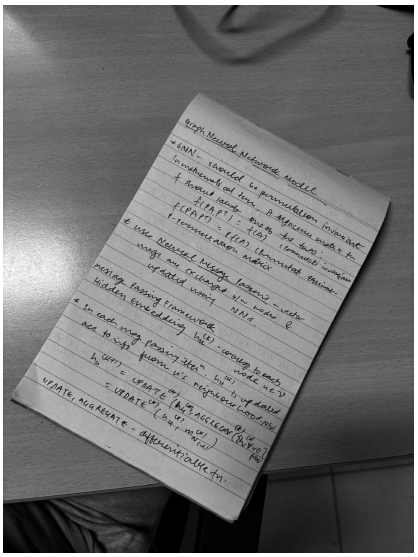
Short explanation. Gamma correction applies the nonlinear mapping $O = 255\,(I/255)^{\gamma}$. In our experiments (e.g. $\gamma = 0.6$) gamma correction was used to enhance textual contrast in shadowed areas without fully clipping highlights; note it can also amplify noise, so it is most effective when combined with a subsequent contrast stretch or denoising step.



(a) $\gamma = 0.6$



(b) $\gamma = 1.0$



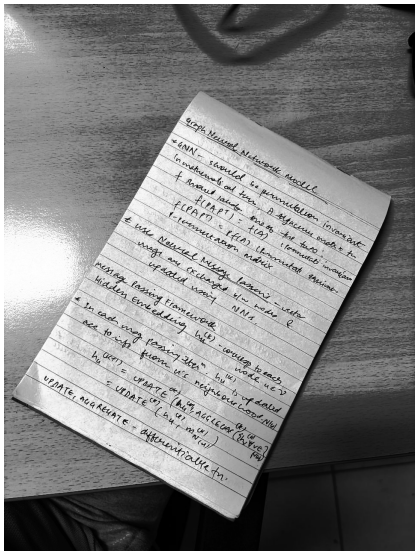
(c) $\gamma = 1.6$

2.4 D. Histogram equalization

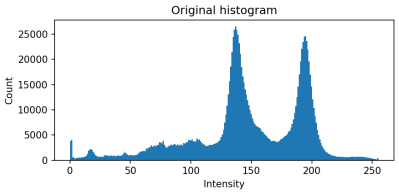
Spreads intensities to use full range; see histogram flattening effect.

Observed result on our image. Comparing original histogram and equalized histogram. The original histogram shows two dominant modes (background and ink) with additional midtones caused by lighting varia-

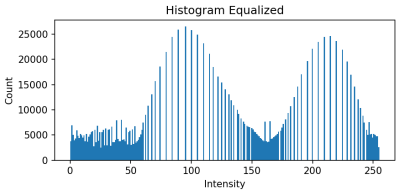
tions. After equalization the intensities are spread, but the result remains strongly non-uniform: two prominent peaks appear at new intensity locations and the histogram shows tall, narrow spikes with many empty bins in between. In visual terms this means that equalization redistributed intensities but did not produce a clean white paper + black text separation; instead it amplified mid-level variations and introduced contrast artifacts in some regions.



(a) Equalized



(b) Original hist



(c) Equalized hist

3 Part 3: Watermarking

3.1 A. Mask extraction

The IITD logo is binarized using a high threshold to obtain a mask $M \in \{0, 1\}$ aligned with the logo.

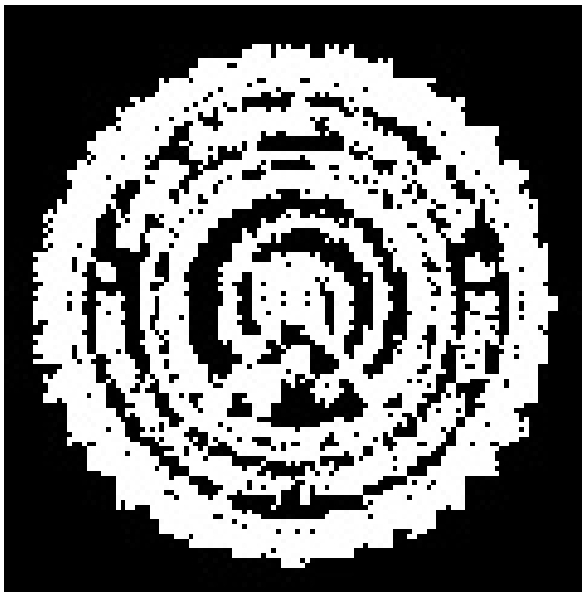


Figure 7: Binary mask of the logo region.

3.2 B. Blending (50% transparency)

With document D, logo L, mask M: $O = D + 0.5\,M\,(L - D) = (1 - 0.5M)D + 0.5M\,L$.

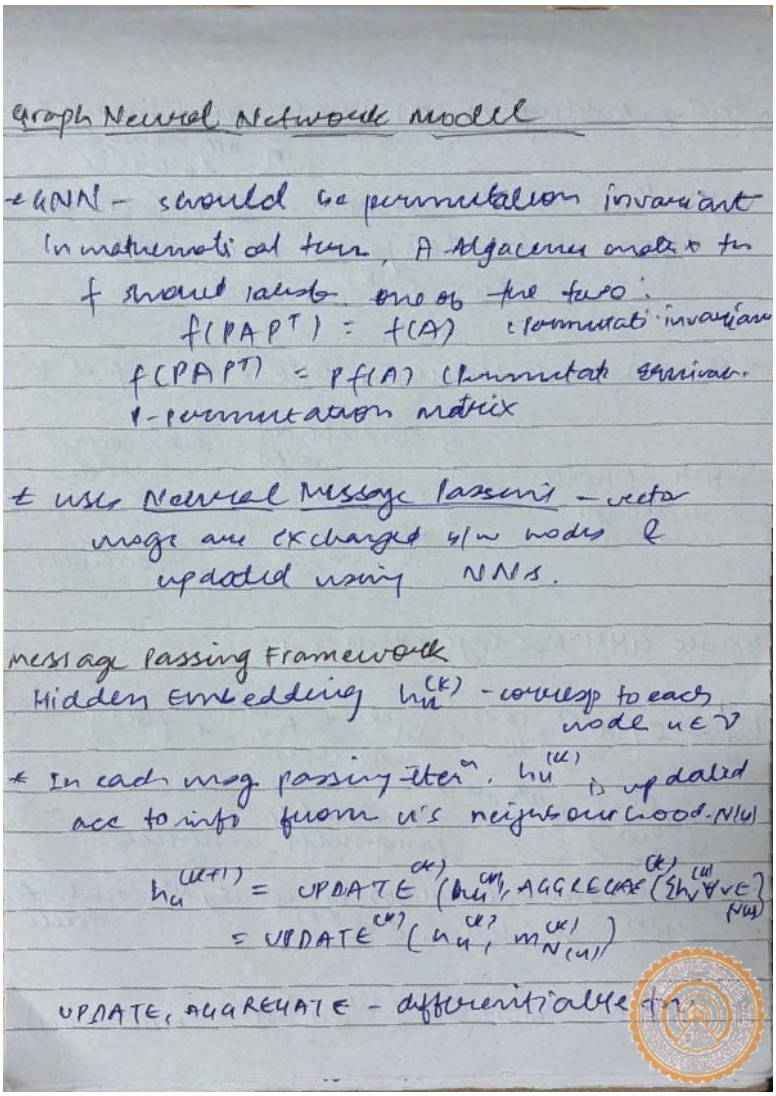


Figure 8: Final watermarked image with 50% transparency.

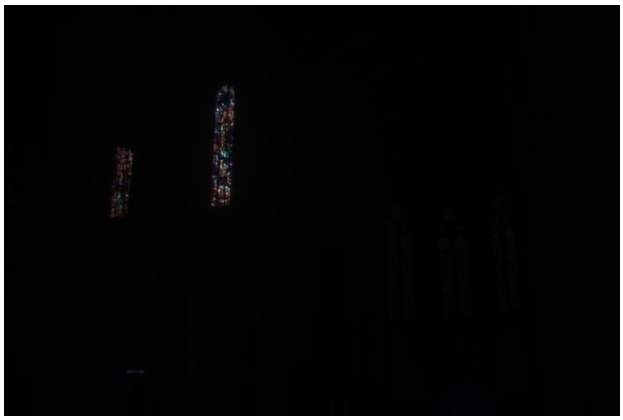
4 Part 4: HDR Histogram Equalization

4.1 A. Algorithm

We equalize HDR intensities without pre-quantization by mapping each channel via its empirical CDF F:

$$s = a + (b - a)\, F(r), \quad \text{with } a{=}0,\ b{=}256.$$

Implementation uses sorting and rank-based CDF with linear interpolation to preserve order and avoid banding.



(a) Original (tonemapped for view)

(b) After equalization

4.2 B. Demonstration (mapping to $[0, 255]$)

Each channel is transformed by $T(r) = a + (b - a)F(r)$ with $(a, b) = (0, 256)$. Since $0 \leq F(r) \leq 1$, it follows that $0 \leq T(r) < 256$. When rendering to 8-bit, values are clipped/quantized to $\{0, \ldots, 255\}$. The outputs in Fig.4(b) visually confirm improved contrast while respecting the target range.

4.3 C. Proofs

Setup. Let R be a real-valued intensity random variable with (possibly channel-wise) CDF $F_R(r) = \mathbb{P}(R \leq r)$. Define the equalization map on $[a, b)$ by

$$T(r) = a + (b - a)\, F_R(r).$$

(C1) Idempotence. Let $S = T(R)$. For $s \in [a, b)$,

$$\mathbb{P}(S \leq s) = \mathbb{P}\big(F_R(R) \leq \tfrac{s-a}{b-a}\big) = \tfrac{s-a}{b-a},$$

so S is uniform on $[a, b)$. Its CDF is $F_S(s) = (s - a)/(b - a)$. Equalizing S gives

$$T'(s) = a + (b - a)F_S(s) = a + (b - a)\tfrac{s-a}{b-a} = s,$$

thus applying histogram equalization twice yields the identity (no further change). The same argument holds for empirical equalization: the map is a (scaled) rank transform, and applying the rank transform to already uniform ranks is the identity.

(C2) Invariance under strictly increasing transforms. Let $g : \mathbb{R} \to \mathbb{R}$ be strictly increasing and define $Z = g(R)$. Then for any x,

$$F_Z(x) = \mathbb{P}(g(R) \leq x) = \mathbb{P}(R \leq g^{-1}(x)) = F_R\big(g^{-1}(x)\big).$$

Equalizing Z at input value $z = g(r)$ yields

$$T_g(z) = a + (b - a)\, F_Z(z) = a + (b - a)\, F_R\big(g^{-1}(g(r))\big) = a + (b - a)\, F_R(r) = T(r).$$

Hence, equalizing after any strictly increasing intensity transform produces the same result as equalizing the original image. In the empirical (discrete) case, g preserves order, so ranks—and therefore the equalized outputs—are identical up to ties.

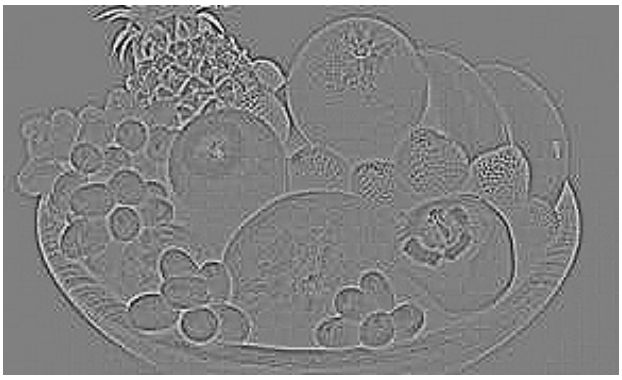
5 Part 5: Convolution

5.1 A. Laplacian (edge emphasis)

Using kernel

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

with additive constant c=128 for visualization. The convolution was implemented with **zero-padding** (i.e. pixels outside the image are treated as zero). This ensures that the output has the same spatial dimensions as the input image.



(a) Laplacian + 128

(b) Mean 3×3 (reference smoothing)

5.2 B. Gaussian: naive 2D vs separable 1D+1D

Gaussian smoothing was tested with $\sigma = 2$. The naive implementation uses a full 2D kernel, while the separable method applies two 1D kernels sequentially (horizontal + vertical). Separability reduces computational complexity from $\mathcal{O}(k^2)$ per pixel to $\mathcal{O}(2k)$.

Both results are visually equivalent but the timing confirms the efficiency gain:

Naive 2D runtime: 1565.02 ms, Separable runtime: 283.40 ms

(a) Naive 2D ($\sigma = 2$)

(b) Separable ($\sigma = 2$)

5.3 C. Laplacian of Gaussian (three ways)

We computed the Laplacian of Gaussian (LoG) in three mathematically equivalent ways:

1. $(L * G) * f$: convolving Laplacian and Gaussian kernels first, then applying to f,
2. $L * (G * f)$: Gaussian smoothing followed by Laplacian,
3. $G * (L * f)$: Laplacian first, then Gaussian smoothing.

Due to discrete padding and rounding to 8-bit values, the three results differ slightly in practice. Theoretically they coincide in the continuous, infinite-domain case.

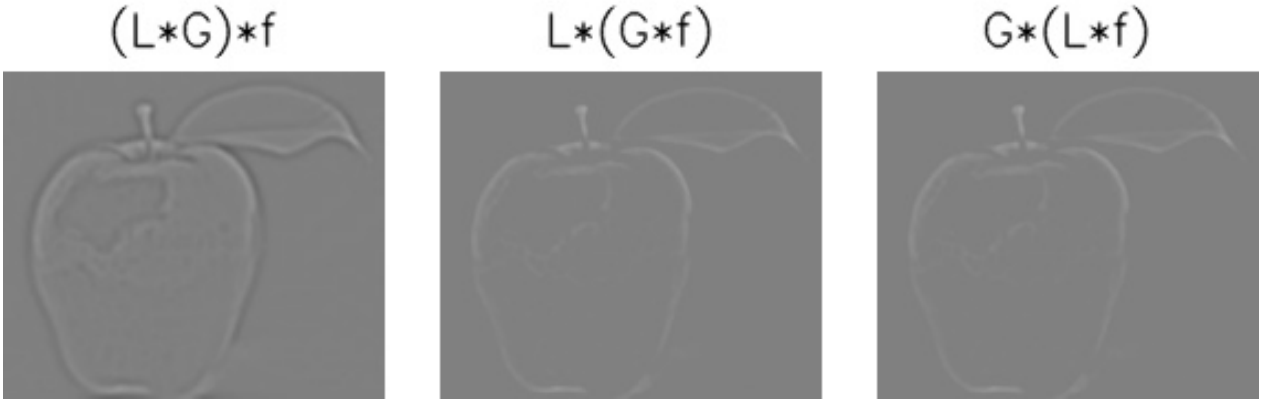


Figure 12: Three LoG variants, laid out with labels.

6 Part 6: Intensity Transform vs Convolution

6.1 A. Linear case

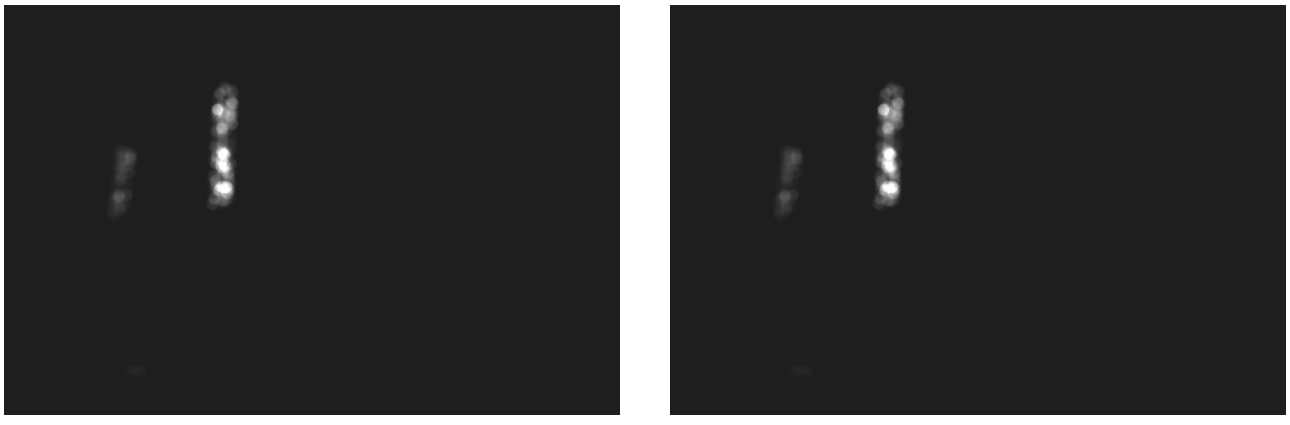
Let $T(r) = ar + b$ and let w be any convolution kernel (not necessarily summing to 1). Define

$$T'(r) = a\,r + b\,S, \qquad S = \sum_{x,y} w(x,y).$$

Then

$$T'(w * f) = a\,(w * f) + b\,S = w * (af) + b\sum w = w * (af + b) = w * T(f).$$

Thus $T'(w * f) = w * T(f)$ holds for all a, b and any w with finite sum S. We verify this numerically in p6 (see images below).



(a) $w * T(f)$ (b) $T'(w * f)$

Figure 13: Part 6A: Linear case equivalence demonstrated on the HDR image.

6.2 B. Nonlinear gamma vs blur

Take T to be gamma with rescaling to [0,1], i.e., $T(r) = (r')^\gamma$ after robust normalization $r' \in [0, 1]$. Let w be a normalized disk kernel of radius r (sum to 1). We compare

$$\text{Gamma} \to \text{Blur}: \quad w * T(f) \quad \text{vs} \quad \text{Blur} \to \text{Gamma}: \quad T(w * f).$$

Because gamma is nonlinear, $w * T(f) \neq T(w * f)$ in general. Blurring first (then compressing dynamic range) tends to mimic an out-of-focus capture, whereas gamma first can preserve micro-contrast before blur. Empirically, the blur-then-gamma result appears closer to an optically defocused image.

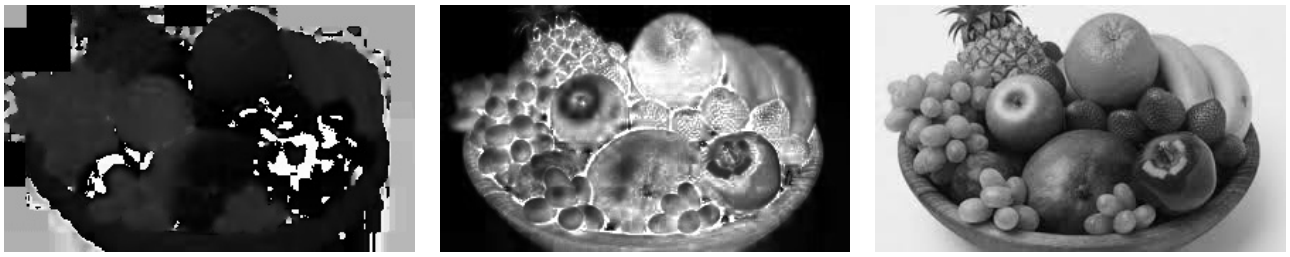
(a) Gamma only (b) $w * T(f)$ (Gamma then Blur) (c) $T(w * f)$ (Blur then Gamma)

Figure 14: Part 6B: Nonlinear order-of-operations comparison.

7 Part 7: Color slicing / HSI

7.1 A. HSI channels

We convert the input RGB image into an HSI color space and display the three channels separately.



(a) Hue (b) Saturation (c) Intensity

7.2 B. Masks and comparison

Colour-slicing principle. Colour slicing selects pixels whose colour vectors lie within a cuboid (or hyperbox) centred at a chosen seed colour. We implement this in two spaces:

1. **RGB cuboid:** select pixels $p = (R, G, B)$ such that $|R - R_s| \leq \Delta_R$, $|G - G_s| \leq \Delta_G$, $|B - B_s| \leq \Delta_B$.
2. **HSI cuboid (practical variant):** select pixels for which:
   - Angular hue distance $d_H = \min(|H - H_s|, 360^\circ - |H - H_s|) \leq \Delta_H$,
   - Saturation $|S - S_s| \leq \Delta_S$,
   - Intensity $|I - I_s| \leq \Delta_I$ (optional).

Pseudocode (mask creation).

```
# seed = pixel at chosen seed location
# For RGB mask:
mask_rgb = (abs(R - seed.R) <= dR) &
           (abs(G - seed.G) <= dG) &
           (abs(B - seed.B) <= dB)

# For HSI mask:
# compute angular hue distance:
dh = abs(H - seed.H)
dh = min(dh, 360 - dh)
mask_hsi = (dh <= dH) & (abs(S - seed.S) <= dS)
# optionally refine with intensity:
mask_hsi = mask_hsi & (abs(I - seed.I) <= dI)
```

Comparison (observed behaviour).

- **RGB mask** is simple but sensitive to intensity variations: shadows, highlights, and shading change RGB channels and may cause holes or false positives.
- **HSI mask** better isolates chromaticity and is therefore more robust to brightness changes. However, it can fail where saturation is low (pale regions of the object appear grey and are missed).
- In practice we found the HSI-based mask to include more of the coloured object while excluding background; small gaps near shadowed regions were fixed by relaxing Δ_I or performing a small morphological closing.

(a) RGB-based mask

(b) HSI-based mask

(c) Side-by-side comparison

7.3 C. Transformations

Additive and multiplicative hue rotations applied to masked regions; summary montage below.

Design goals. A realistic recolouring should change the object's perceived hue while preserving texture, shading and natural appearance. This means (a) handling hue as an angular quantity, (b) avoiding abrupt saturation/intensity changes that flatten texture, and (c) applying transformations only on masked pixels (or using a soft mask to blend).

Recommended transform forms. Let a source pixel in HSI be $c = (H, S, I)$ and the target colour be $c_t = (H_t, S_t, I_t)$. Common choices:

- **Hue rotation (additive modulo 360):**

$$H' = (H + \Delta_H) \bmod 360,$$

where $\Delta_H = H_t - H_s$ (often computed from a seed colour H_s). Hue must be handled with wrap-around; this is additive (rotation) not multiplicative.

- **Saturation scaling (multiplicative):**

$$S' = \text{clip}(\alpha_S \cdot S,\, 0, 1),$$

where $\alpha_S > 0$ scales colour purity. Multiplicative change preserves relative differences in saturation across the object.

- **Intensity change (multiplicative or small additive):**

$$I' = \text{clip}(\alpha_I \cdot I + \beta_I,\, 0, 255),$$

where multiplicative α_I preserves shading contrast (preferred). Small additive β_I can be used for uniform brighten/darken but may wash out texture if large.

Full per-pixel transform (applied only for mask pixels).

$$T(H, S, I) = \big((H + \Delta_H) \bmod 360,\ \text{clip}(\alpha_S S, 0, 1),\ \text{clip}(\alpha_I I + \beta_I, 0, 255)\big).$$

For a soft mask $w(x, y) \in [0, 1]$ blend between original and transformed HSI (or convert back to RGB and blend there):

$$C_{\text{out}} = (1 - w)\, C_{\text{orig}} + w\, C_{\text{transformed}}.$$

Why additive for hue and multiplicative for S,I? Hue is an angular attribute — shifting colour is naturally expressed as an additive rotation on the angle. Saturation and intensity represent magnitudes; multiplicative scaling preserves relative contrast and texture (e.g. shadows remain darker than highlights after scaling). A vector-style multiplicative transform $(c_t/c_s)\cdot c$ is not appropriate because hue is angular and division is undefined when any component is near zero; it also tends to distort chromatic relationships.

Implementation notes and examples.

```
# Given mask (binary or soft) and transform params:
delta_H = target_H - seed_H
alpha_S = desired_saturation_scale  # e.g., 1.2 to increase
alpha_I = desired_intensity_scale   # e.g., 1.05
beta_I  = desired_intensity_offset  # e.g., 0

for each pixel p:
    if mask[p] > 0:
        H' = (H[p] + delta_H) % 360
        S' = clip(alpha_S * S[p], 0, 1)
        I' = clip(alpha_I * I[p] + beta_I, 0, 255)
        out[p] = HSI_to_RGB(H', S', I')
    else:
        out[p] = orig[p]
# For soft masks, blend out[p] with orig[p] by mask weight.
```

Qualitative outcomes. Using hue rotation with mild saturation scaling and small multiplicative intensity adjustment produces realistic recolouring: the object acquires the new hue while preserving internal shading and texture. Aggressive saturation or intensity changes produce a synthetic, flattened look. When the object contains low-saturation regions, consider applying a reduced hue shift there or increasing saturation first.

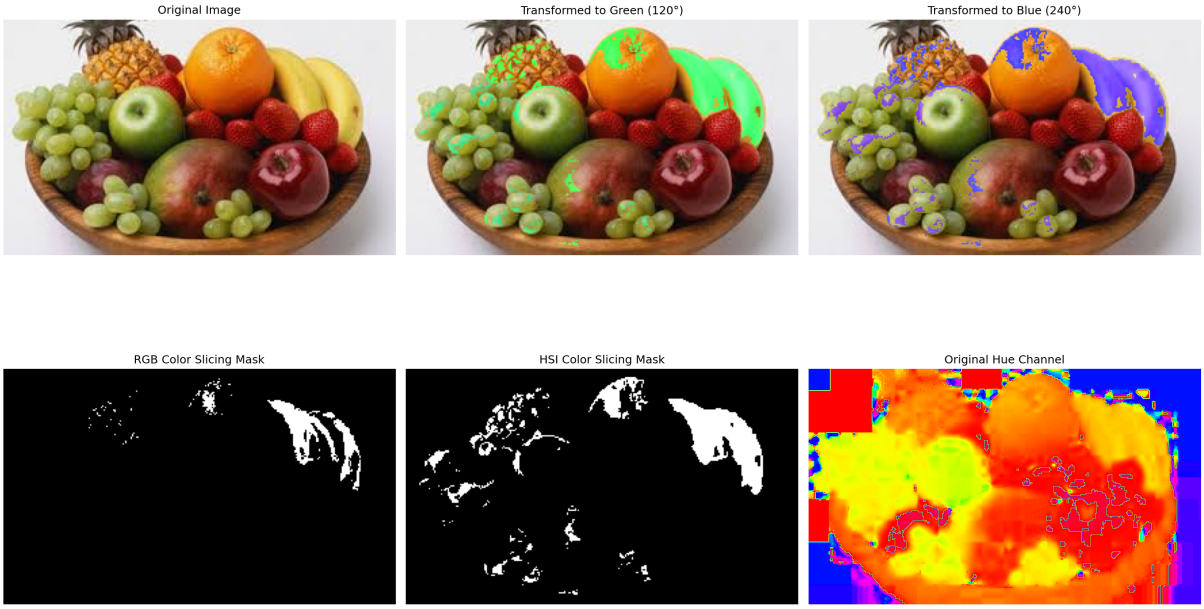


Figure 17: Representative color-slicing transformations.